

Design Principles and Guidelines for LLM Observability: Insights from Developers

Xin Chen
Alibaba Cloud Computing
Alibaba Group
Hangzhou, China
pola.cx@alibaba-inc.com

Yan Li
Alibaba Cloud Computing
Alibaba Group
Hangzhou, China
ruoxi@alibaba-inc.com

Xiaoming Wang
Alibaba Cloud Computing
Alibaba Group
Hangzhou, China
xiaoming.wxm@alibaba-inc.com

Abstract

The growing number of large model development tools has increased the demand for transparency and accountability. We present four developer-centric design principles for large language model (LLM) observability across the entire LLMOps lifecycle. The research focuses on improving the developer experience by integrating design principles that address the unique needs of developers at various stages of LLM development. A mixed-methods methodology was implemented consisting of three focus groups (n=10 per group) with developers across different proficiency levels, complemented by a small-scale designer rating survey. The study highlights key design principles—Design for Awareness, Monitoring, Intervention, and Operability—and their impact on developer experience factors like mental models, efficiency, visualization, and understandability. The findings reveal significant correlations between these principles and developer needs, with a particular trade-off between efficiency and mental model clarity. The study provides actionable insights for designing effective observability tools and emphasizes the importance of balancing developer control, transparency, and ease of use.

CCS Concepts

• **Human-centered computing**; • **Human computer interaction (HCI)**; • **HCI theory, concepts and models**;

Keywords

LLM Observability, Design Principles, Developer Experience

ACM Reference Format:

Xin Chen, Yan Li, and Xiaoming Wang. 2025. Design Principles and Guidelines for LLM Observability: Insights from Developers. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3706599.3719914>

1 INTRODUCTION

Large Language Models (LLMs) have gained significant attention in the tech community due to their natural language generation capabilities. However, they present challenges like hallucinations [1],

security risks [2], high costs [3], and unpredictable performance [4]. Issues like prompt injection [5] attacks and reliance on third-party APIs [6] further complicate their reliability. To address these challenges, robust LLM observability is essential, focusing on security, reliability, and ethical use.

LLM observability [7-9] is the process of monitoring and tracking the performance of LLMs and LLM outputs in real time. It involves keeping an eye on how well the model is working, detecting errors, and ensuring it behaves as expected [10]. LLM observability tools are specific platforms that support the achievement of the broader goal of LLM observability. Developers leverage these tools to collect, process, and analyze LLM operational data, thereby monitoring [9] and optimizing model behavior in real-time [11] during the entire LLMOps lifecycle [12-15]. Various LLM observability tools have emerged in the market, such as Datadog, Arize, OpenTelemetry and Wandb.

Over the past four decades, Human-Computer Interaction (HCI) research has established a robust foundation of guidelines and frameworks for developing effective and safe computing systems. These include widely applicable principles such as user interface design heuristics [16] and human-computer interaction strategies [17], as well as technology-specific guidelines like web usability standards [18] and human-AI interaction frameworks [19, 20]. However, the emergence of LLMs presents unprecedented challenges that existing HCI guidelines fail to address adequately. As LLM technologies continue to evolve rapidly, traditional design principles prove insufficient in explaining the complex interactions between developers and LLM systems. Furthermore, conventional research findings may not fully apply to the increasingly diverse developer community working with LLMs today.

This study explores developer-centric design principles for enhancing LLM observability across the entire LLMOps lifecycle, with the goal of catering to developers of varying expertise levels and optimizing their workflow. The main contributions of this work are threefold: (i) introducing four developer-centric design principles for LLM observability, (ii) validating these principles through comprehensive user studies with developers and design teams, and (iii) providing practical guidelines for implementing these principles in real-world applications.

2 RELATED WORK

In recent years, the increasing complexity and widespread use of LLMs have driven extensive efforts to evaluate transparency and interpretability in AI systems.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/2025/04

<https://doi.org/10.1145/3706599.3719914>

2.1 Explainable AI and LLM Observability

Explainable AI (XAI) research aims to enhance the interpretability and transparency of AI systems [21-25]. Although human-centered XAI has been widely explored [26, 27], assessing the quality of explanations remains a challenge. Current evaluation methods include functionally-grounded, application-grounded, and human-grounded approaches [28], with the latter two requiring human participation. While automated evaluation measures for XAI algorithms exist, they face issues with comparability and may fail to accurately capture human preferences [29-33]. Despite the importance of user studies in XAI evaluation, only 20% of XAI projects involve human subjects [30]. Moreover, as XAI has developed differently within various communities, such as machine learning and HCI [34], there is an urgent need for effective guidance on designing user studies to help designers better understand users' needs [35].

XAI addresses the “why” of model decisions, explaining which inputs influenced the outcome. LLM observability, on the other hand, provides insights into the “what” and “how” of model behavior, monitoring its performance and operational health throughout its lifecycle [9, 36]. While both share a focus on understanding model behavior, XAI targets individual decisions, whereas LLM observability covers broader aspects like model performance, reliability, and long-term optimization [37]. Thus, they are strongly connected but serve distinct purposes.

2.2 User Needs and AI Expertise Characteristics in XAI Design

Traditional XAI approaches focus on technical transparency, whereas human-centered design paradigms prioritize user engagement and the practicality of explanations. Previous research has identified three key categories of XAI users, each with distinct needs and expertise: AI novices (general AI product end-users), data experts (domain and data analytics specialists), and AI experts (machine learning model designers) [37]. To help end users understand, trust, and meet their needs, HCI and AI researchers have developed innovative, user-centered algorithm visualizations, interfaces, and toolkits [38-40].

Studies have emphasized that XAI systems should be tailored to user classification, as the effectiveness of these systems improves when designed to suit users' specific needs and expertise levels [20]. However, current XAI research predominantly targets technical staff and overlooks usability for non-expert users. More human-centered approaches are needed, as existing studies often lack real-world task evaluations and fail to address the unique needs of lay users [11, 41].

Overall, these studies often overlook both the practical needs and developer experience in LLM operations. This study fills a critical gap in LLM observability by addressing the user-centric challenges faced by developers at various expertise levels, which are often neglected in existing frameworks that primarily emphasize technical metrics [42].

3 DESIGN PRINCIPLES AND GUIDELINES FOR LLM OBSERVABILITY

This section begins by examining the principles that should guide our proposed framework (Section 3.1), then proceeds to discuss how to build this comprehensive framework (Section 3.2).

3.1 Developer Experience

Within the developer experience (DX) framework [43, 44], research has identified several key factors that significantly influence developers' perceptions, feelings, and values in software development [45]. Empirical studies have highlighted factors that affect cognitive patterns [46] and productivity [47] in software development. Based on these findings, we focus on four factors below that are particularly relevant to LLM development.

Mental Models: Developers require XAI systems to help form accurate mental models of AI's decision-making process. The ability to understand how an AI model works improves developers' ability to predict its behavior. The need for accurate mental models is emphasized across various research [48-50], showing that developers need clarity in how the model functions and reasons about data. This mental framework helps in better debugging, fine-tuning, and improving the AI systems.

Understandability: For effective interaction with AI systems, developers seek transparent systems that enhance their understanding. The explanation tools provided by XAI systems should support developers in grasping the underlying logic of decisions and predictions made by the models. Explanations that clarify model reasoning can foster trust and reduce cognitive load, enabling developers to make informed decisions during the development and deployment phases.

Visualization: Developers demand visual tools that can represent AI models and their predictions clearly. Visualization of complex models such as neural networks helps in interpreting outputs and debugging models. Tools like saliency maps or decision trees are particularly useful for understanding which features influence model behavior and ensuring that developers can interact with and refine their models effectively.

Efficiency: Developers expect XAI systems to improve task efficiency by simplifying debugging and monitoring processes. The integration of real-time explanations and performance feedback allows developers to quickly identify and resolve issues, enhancing productivity. Efficient debugging mechanisms, such as explanations that clarify why specific outputs are generated, are essential for optimizing model performance and saving time during the development cycle.

Later, to validate the connection between design principles and user experience, we will analyze them based on these four key factors (Section 4.2).

3.2 Framework Organization

Next, we conducted three focus groups with developers of different expertise levels to collect feedback on LLMops practices (Section 4.1). From developer feedback, we established a framework comprising four stages, with the complete developer journey illustrated in Figure 1. The resulting analysis informed our four design principles and associated guidelines, which are presented in Table 1. The

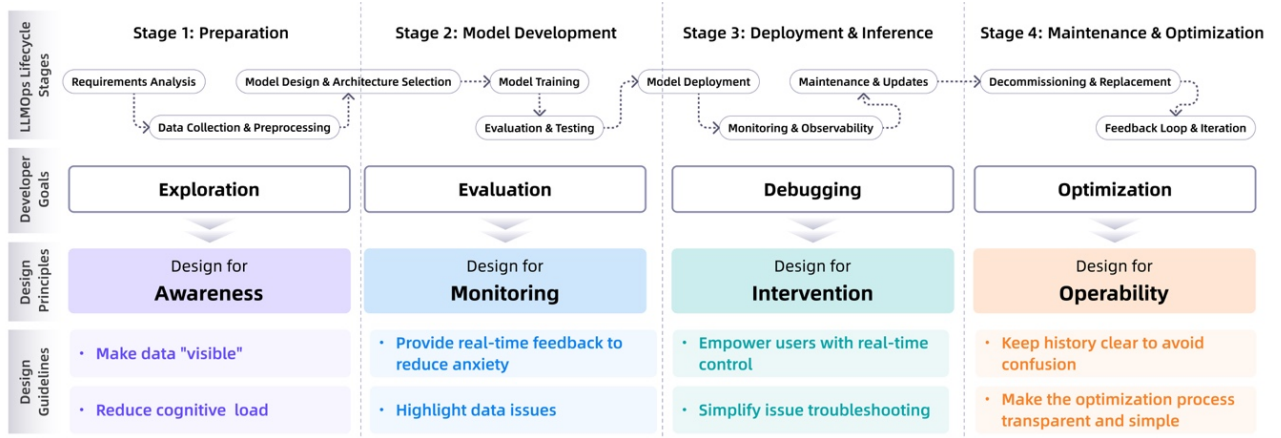


Figure 1: End-to-End Developer Journey and Design Principles and Guidelines Across LLMOps Lifecycle

rest of this paper explains the methodology used to develop and validate these principles and guidelines.

These principles should be viewed as general guidelines rather than mandatory requirements for all design work. We believe designers should rely on their professional judgment to evaluate which principles fit their specific circumstances and whether certain guidelines are worth implementing.

4 METHODOLOGY

Our goal is to produce a set of clear, concise, and relevant design principles that can be readily applied by designers in the design of tools that incorporate LLM observability technologies. We used a mixed-methods approach to construct and validate these principles.

4.1 Insights from Developers

4.1.1 Focus Group. We conducted three focus groups with a total of 30 developers from an anonymous LLM provider. Each group included 10 participants from junior, intermediate, and senior levels. The goal was to explore LLM development practices. These developers have varying levels of development experience and have used different LLM development platforms or tools, such as Hugging Face, Google Cloud AI Platform, and Azure AI Studio. The interview framework covered the entire LLMOps lifecycle, including data preparation, model development, deployment, and maintenance. As shown in Table 2, we focused on understanding the technical challenges faced by developers, their tool usage patterns, requirements for observability tools, and how different feedback mechanisms impact development efficiency.

4.1.2 Analysis. Next, we collected feedback from developers with varying skill levels through focus groups about their LLM development process. Using card sorting, we categorized the model metrics and tool features that developers prioritized to identify their needs and pain points. The collected data, as presented in Table 3, provides empirical evidence to characterize the challenges and requirements in the LLMOps process.

4.2 Validating the Significance of Different Design Principles

4.2.1 Survey. We aimed to quantify the correlations between key design principles (Design for Awareness, Monitoring, Intervention, Operability) and developer experience factors (Mental Models, Efficiency, Visualization, Understandability). To achieve this, we recruited 10 experienced designers specializing in large model platform design. They rated the correlations between the design principles and DX factors. Then we used statistical methods including descriptive statistics and factor analysis to validate the significance of each design principle.

4.2.2 Data Analysis. Figure 2a reveals that Design for Awareness demonstrated the strongest correlation with Mental Models (0.88) and Visualization (0.92), emphasizing its critical role in enhancing user comprehension and the interpretability of information. However, its moderate impact on Efficiency (0.52) suggests that awareness-driven designs may not prioritize operational performance. Conversely, Design for Intervention showed a strong positive correlation with Efficiency (0.88) but a negative correlation with Mental Models (-0.20), indicating that while real-time interventions streamline workflows, they may disrupt user understanding of the system's behavior. Design for Operability balanced Visualization (0.56) and Efficiency (0.82), supporting its applicability in production environments requiring scalable and intuitive operations.

The trade-off between Efficiency and Mental Models is particularly noteworthy (Figure 2b). Intervention-based designs enhance task performance but can overwhelm users, whereas Awareness-driven designs strengthen understanding at the potential cost of productivity. These findings align with existing literature highlighting the role of clear contextual information in cognitive modeling but also reveal conflicting evidence on intervention-driven workflows, suggesting potential cognitive overload.

Future research should investigate these principles in domain-specific and longitudinal contexts to refine their application. Additionally, personalized observability tools could mitigate trade-offs, ensuring balanced improvements in both efficiency and user comprehension. This study provides actionable insights for advancing

Table 1: Design Principles and Guidelines for LLM Observability.

Developers Goals	Design Principles	Design Guidelines
Exploration	Design for Awareness Ensure developers can clearly perceive model capabilities, effectiveness, pricing, and other attributes, laying the foundation for model development	Make data “visible” Facilitate an intuitive understanding of data distribution, quality, and potential issues through visual representation. • <i>Comprehensive model information display.</i> Provide a clear view of the entire dataset, its features, and overall characteristics, such as multimodal type, business scenario, version updates, and billings. • <i>Task-Guided Data Preparation.</i> Provide preconfigured data cleaning or augmentation workflows tailored to task-specific needs (e.g., classification or generation).
		Reduce cognitive load Minimize manual data inspection by providing simple and user-friendly tools. • <i>Performance Comparison Visualization.</i> Use radar or distribution plots to compare model performance across different models or datasets, enabling developers to evaluate effectiveness effortlessly. • <i>Data summaries and statistical insights.</i> Present high-level data summaries to give developers a quick understanding of the data they working with.
Evaluation	Design for Monitoring Help developers monitor training processes and model performance in real-time, quickly identify issues, and optimize the development experience	Provide real-time feedback to reduce anxiety Provide live updates on model progress to help developers avoid extended periods of uncertainty. • <i>Real-Time Training Progress Display.</i> Dynamically update training loss curves and provide contextual cues (e.g., “Model is converging”) to alleviate uncertainty. • <i>Anomaly Warning Cards.</i> Use cards to highlight potential issues like overfitting (e.g., declining validation accuracy) or gradient explosions.
		Highlight data issues Specifically focuses on drawing attention to problematic data points or issues that might affect the model’s performance or the integrity of data. • <i>Data Visualization Panel.</i> Present data distribution using simple charts such as bar charts and pie charts to help developers in viewing a rich set of metrics and identifying anomalies. • <i>Automated Issue Detection.</i> Automatically flag duplicated, noisy, or anomalous data, and highlight areas that require developer attention.
Debugging	Design for Intervention Help developers monitor training processes and model performance in real-time, quickly identify issues, and optimize the development experience	Empower users with real-time control Enable developers to intervene and adjust model outputs. • <i>Adjustable Parameters with Immediate Feedback.</i> Allow users to adjust key parameters (e.g., thresholds, learning rates, or weights) and immediately observe the impact on outputs. Example: Provide sliders for modifying AI model parameters like “temperature” or “top-k” in natural language generation tasks, with real-time output updates. • <i>Output Correction Tool.</i> Offer a real-time editing interface that allows developers to adjust or suppress unsuitable outputs (e.g., sensitive terms or erroneous content). Example: Developers can directly click on problematic prompts to “replace” or “delete” them.
		Simplify issue troubleshooting Provide clear context for problematic outputs, reducing the effort required to trace issues. • <i>Input-Output Comparison Panel.</i> Display the relationship between inference inputs and outputs, marking potential issues such as repetitive outputs or incoherent logic. Example: Highlight outputs that are “unrelated to the input text.” • <i>Multi-level debugging and dynamic data pipeline visualization.</i> Provide detailed logs for deeper investigation (e.g., showing which input data caused the high latency). And allow users to drill-down interactively to inspect affected data, layers, or parameters.

Optimization	Design for Operability Provide long-term maintainability and efficient operations of models, reduce developer burden, and ensure models consistently maintain optimal performance	Keep history clear to avoid confusion Help developers quickly understand model changes and their impacts. • <i>Model Version Tracking and Comparison Tool.</i> Display changes and performance differences across model versions through visualizations (e.g., accuracy comparison charts). • <i>Visual Timelines and Time-traced Logs.</i> Allow developers to track key modifications, issues, and review development changes like preprocessing, fine-tuning, and deployment with expandable details.
		Make the optimization process transparent and simple Provide streamlined tools and guidance for complex optimization tasks. • <i>Continuous Learning Monitoring Panel.</i> Monitor model performance in production environments, generate performance reports, and suggest optimizations (e.g., “Accuracy for long-tail classes has declined”). Example: Notify developers about insufficient data in specific domains of a new dataset. • <i>Automated Resource Optimization Assistant.</i> Provide one-click optimization features, such as model distillation and pruning, to reduce inference costs. Show real-time progress and estimated optimization benefits during operations.

Table 2: Interview Objectives for Three Developer Skill Levels

Developer Skill Levels	Working Years	Sample Size	Interview Objectives
Junior	<1 year	10	Understand the diverse challenges, preferences, and tool requirements that developers encounter when developing and monitoring Large Language Models (LLMs). Analyze the complex issues encountered by developers with some experience in LLM development and their expectations for tool functionalities. Explore developers’ needs and recommendations regarding large-scale deployment, performance optimization, and maintenance of LLMs.
Intermediate	1-3 years	10	
Senior	>3 years	10	

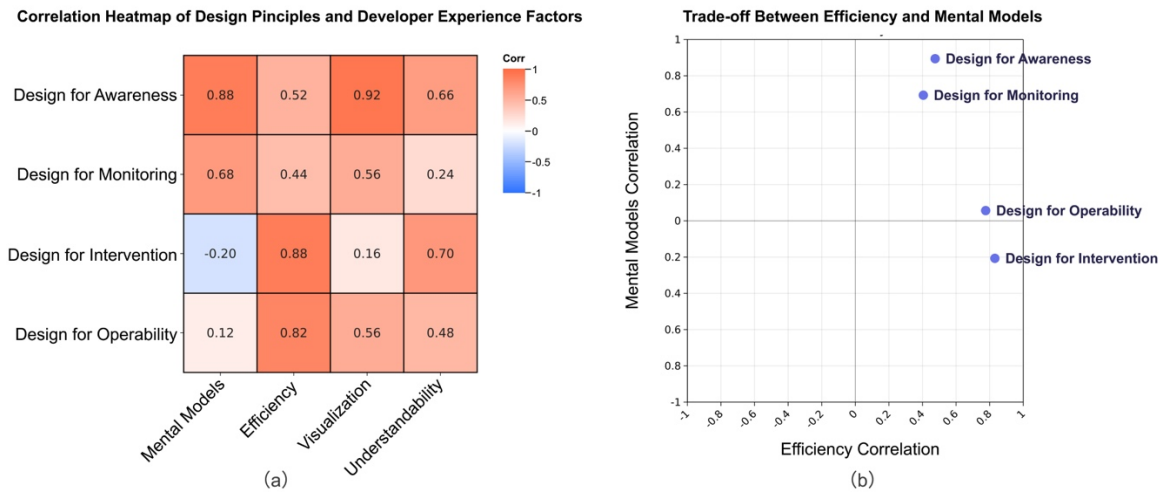


Figure 2: Evaluators’ ratings of the (a) relevance and (b) distribution of each principle between efficiency and mental models.

Table 3: Challenges, Key Needs and Representative Quotes from Focus Group Participants

Developer Skill Levels	Challenges	Key Needs	Representative Quotes
Junior	Onboarding issues and Data Preparation	Intuitive interfaces Pre-configured workflows	“Which model would be the most appropriate for my business requirements, and what options are available to me?” “I often struggle to see if my data distribution aligns with the task requirements.” “It’s hard to see if my dataset is balanced without additional tools.” “I don’t know if my data quality is good enough until the model fails.” “There’s too much information in monitoring dashboards. I don’t know where to start.”
Intermediate	Expectations for Monitoring Tools	Customizable dashboards Real-time performance monitoring	“I need to know how the loss and accuracy are evolving without digging through logs.” “I want alerts to tell me exactly where the issue lies, not just that something is wrong.” “During training, I want to know if the model is overfitting as soon as it happens.” “I often spend hours debugging why validation accuracy suddenly drops.”
Senior	Long-Term Maintenance	Integration with AIOps tools Advanced customization	“Tracking performance changes across versions is essential when managing models in production.” “I need to understand how different versions of the model perform over time.” “Tracking model drift over six months is difficult without robust tools.”

lifecycle-wide, developer-centric design frameworks for LLM observability.

5 DISCUSSION

The findings of this study have significant implications for the design of developer-centric observability tools across the entire LLMOps lifecycle. By focusing on the needs and challenges faced by developers, the study emphasizes the importance of designing observability systems that are intuitive, actionable, and adaptable to varying levels of expertise.

In terms of theoretical contributions, the research introduces a developer-centric approach that bridges the gap between technical observability metrics and the real-world usability challenges developers face at each stage of the LLMOps. This approach offers a more comprehensive framework for LLM observability, moving beyond abstract performance metrics to address the practical demands of developers, such as ease of use, real-time control, and efficient debugging. The study also presents a detailed analysis of the trade-offs between different design principles (e.g., Design for Awareness vs. Design for Intervention), which will contribute to future theoretical frameworks on the balance between efficiency and cognitive load.

From a practical standpoint, the proposed design principles for LLM observability can be implemented in real-world tools through several technical pathways. For Design for Awareness, data lineage visualization can be integrated into data dashboards, similar to

Hugging Face’s Datasets library, providing dynamic distribution analysis and metadata tracking. To improve LLM observability tools, TensorBoard could integrate XAI layers for clearer real-time explanations and support dynamic parameter adjustments. Similarly, Wandb would benefit from additional guidance on data pre-processing, along with a task model library and automated data cleaning. These are merely suggestions and should be evaluated and validated based on specific features and user needs.

Policy implications include the potential for improved regulatory compliance and ethical AI [51] development. By focusing on transparency and accountability, developer-centric design principles can support the broader goals of responsible AI [52] deployment. Policymakers could use these insights to promote best practices in AI system design, ensuring that developers have access to tools that prioritize both performance and usability.

Despite the contributions of this study, several limitations exist. The small sample size of 30 developers limits the generalizability of the findings. Additionally, the proposed design principles have not been fully tested in real-world large-scale applications. Future work should involve broader, diverse samples, real-world validation in production environments, and a deeper exploration of the technical implementation challenges to enhance the applicability and scalability of these principles across various LLM architectures and domains.

6 CONCLUSION

We have innovatively proposed a developer-centric design approach, systematically placing developer needs and challenges at the core of LLM observability tool design, bridging the gap between technical metrics and real-world development requirements. The study adopts a lifecycle-spanning perspective, covering all stages from data preparation to model deployment and maintenance, ensuring that the design principles address the needs at each phase. Four key design principles—“Design for Awareness,” “Design for Monitoring,” “Design for Intervention,” and “Design for Operability”—are introduced, along with specific design guidelines to guide developers in building intuitive, efficient, and user-friendly LLM observability systems. Additionally, the research delves into the trade-offs between different design principles and their impact on efficiency and cognitive load. Empirical evidence is provided to validate the effectiveness and application potential of these principles, particularly in enhancing developer efficiency, improving system transparency, and reducing cognitive overload.

References

- [1] Jim Waldo, and Soline Boussard. 2024. GPTs and Hallucination: Why do large language models hallucinate? Queue 22, 4, <https://doi.org/10.1145/3688007>
- [2] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2025. Security and Privacy Challenges of Large Language Models: A Survey. ACM Comput. Surv. 57, 6, <https://doi.org/10.1145/3712001>
- [3] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qishui Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jia Shi Li, Wenfeng Liang, Xiaodong Liu, Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, Junjie Qiu, Hui Qu, Zehui Ren, Zhangli Sha, Xuecheng Su, Xiaowen Sun, Yixuan Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shuiping Yu, Yukun Zha, Liyue Zhang, Haowei Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. 2024. Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning. In Proceedings of the Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, 2024, Atlanta, GA, USA. IEEE Press, Article 83. <https://doi.org/10.1109/sc41406.2024.00089>
- [4] Samuel Greengard. 2024. Is It Possible to Truly Understand Performance in LLMs? Commun. ACM 67, 12, <https://doi.org/10.1145/3695860>
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In Proceedings of the Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, 2023, Copenhagen, Denmark. Association for Computing Machinery, 79–90. <https://doi.org/10.1145/3605764.3623985>
- [6] Chaozheng Wang, Shuzheng Gao, Cuiyun Gao, Wenxuan Wang, Chun Yong Chong, Shan Gao, and Michael R. Lyu. 2024. A Systematic Evaluation of Large Code Models in API Suggestion: When, Which, and How. In Proceedings of the Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, Sacramento, CA, USA. Association for Computing Machinery, 281–293. <https://doi.org/10.1145/3691620.3695004>
- [7] Tian Yu Liu, Stefano Soatto, Matteo Marchi, Pratik Chaudhari, and Paulo Tabuada. 2024. Meanings and Feelings of Large Language Models: Observability of Latent States in Generative AI. ArXiv abs/2405.14061
- [8] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. ArXiv abs/2411.05285
- [9] P Ganesan. 2024. LLM-Powered Observability Enhancing Monitoring and Diagnostics. J Artif Intell Mach Learn & Data Sci 2024 2, 2.
- [10] Lingfeng Sun, Devesh K. Jha, Chiori Hori, Siddharth Jain, Radu Corcodel, Xinghao Zhu, Masayoshi Tomizuka, and Diego Romero. 2023. Interactive Planning Using Large Language Models for Partially Observable Robotic Tasks. 2024 IEEE International Conference on Robotics and Automation (ICRA)
- [11] Qi Zhang, Jieming Zhu, Jiansheng Sun, Guohao Cai, Ruining Yu, Bangzheng He, and Liangbi Li. 2024. Enhancing News Recommendation with Real-Time Feedback and Generative Sequence Modeling. In Proceedings of the Proceedings of the Recommender Systems Challenge 2024, 2024, Bari, Italy. Association for Computing Machinery, 32–36. <https://doi.org/10.1145/3687151.3687158>
- [12] R.A. R. 2024. Mastering Large Language Models with Python: Unleash the Power of Advanced Natural Language Processing for Enterprise Innovation and Efficiency Using Large Language Models (LLMs) with Python. Orange Education Pvt Limited.
- [13] Josu Diaz-De-Arcaya, Juan López-De-Armentia, Raúl Miñón, Iker Lasa Ojanguren, and Ana I Torre-Bastida 2024. Large Language Model Operations (LLMOps): Definition, Challenges, and Lifecycle Management. IEEE, City.
- [14] Chakkrit Kla Tantithamthavorn, Fabio Palomba, Foutse Khomh, and Joselito Joey Chua 2025. MLOps, LLMops, FMOps, and Beyond. IEEE Computer Society, City.
- [15] Saurabh Pahune, and Zahid Akhtar. 2025. Transitioning from MLOps to LLMops: Navigating the Unique Challenges of Large Language Models. Information 16, 2.
- [16] Jakob Nielsen, and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In Proceedings of the Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 1990, Seattle, Washington, USA. Association for Computing Machinery, 249–256. <https://doi.org/10.1145/97243.97281>
- [17] Ben Shneiderman, Catherine Plaisant, Maxine Cohen, Steven Jacobs, Niklas Elmquist, and Nicholas Diakopoulos. 2016. Designing the User Interface: Strategies for Effective Human-Computer Interaction. Pearson.
- [18] Nigel Bevan 2005. Guidelines and standards for web usability. Citeseer, City.
- [19] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fournay, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In Proceedings of the Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019, Glasgow, Scotland Uk. Association for Computing Machinery, Paper 3. <https://doi.org/10.1145/3290605.3300233>
- [20] Qian Yang, Aaron Steinfeld, Carolyn Rosé, and John Zimmerman. 2020. Re-examining Whether, Why, and How Human-AI Interaction Is Uniquely Difficult to Design. In Proceedings of the Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, 2020, Honolulu, HI, USA. Association for Computing Machinery, 1–13. <https://doi.org/10.1145/3313831.3376301>
- [21] Vikas Hassija, Vinay Chamola, Atmesh Mahapatra, Abhinandan Singal, Divyansh Goel, Kaizhu Huang, Simone Scardapane, Indro Spinelli, Mufti Mahmud, and Amir Hussain. 2024. Interpreting Black-Box Models: A Review on Explainable Artificial Intelligence. Cognitive Computation 16, 1, <https://doi.org/10.1007/s12559-023-10179-8>
- [22] Gesina Schwalbe, and Bettina Finzel. 2024. A comprehensive taxonomy for explainable artificial intelligence: a systematic survey of surveys on methods and concepts. Data Mining and Knowledge Discovery 38, 5, <https://doi.org/10.1007/s10618-022-00867-8>
- [23] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Ben- netot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. 2020. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. Information Fusion 58 <https://doi.org/https://doi.org/10.1016/j.inffus.2019.12.012>
- [24] Prachi Zodge, Hussain Harianawala, Hafsa Shaikh, and Asad Kharodia. 2024. Explainable AI (XAI): History, Basic Ideas and Methods. International Journal of Advanced Research in Science, Communication and Technology
- [25] Wenli Yang, Yuchen Wei, Hanyu Wei, Yanyu Chen, Guan Huang, Xiang Li, Renjie Li, Naimeng Yao, Xinyi Wang, Xiaotong Gu, Muhammad Bilal Amin, and Byeong Kang. 2023. Survey on Explainable AI: From Approaches, Limitations and Applications Aspects. Human-Centric Intelligent Systems 3, 3, <https://doi.org/10.1007/s44230-023-00038-y>
- [26] Mark O. Riedl. 2019. Human-Centered Artificial Intelligence and Machine Learning. ArXiv abs/1901.11184
- [27] Upol Ehsan, and Mark O. Riedl 2020. Human-Centered Explainable AI: Towards a Reflective Sociotechnical Approach. Springer International Publishing, City.
- [28] Finale Doshi-Velez, and Been Kim. 2017. Towards A Rigorous Science of Interpretable Machine Learning. arXiv: Machine Learning
- [29] Peter Hase, and Mohit Bansal 2020. Evaluating Explainable AI: Which Algorithmic Explanations Help Users Predict Model Behavior? , City.
- [30] Meike Nauta, Jan Trienes, Shreyasi Pathak, Elisa Nguyen, Michelle Peters, Yasmin Schmitt, Jörg Schlötterer, Maurice van Keulen, and Christin Seifert. 2023. From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI. ACM Comput. Surv. 55, 13s, <https://doi.org/10.1145/3583558>
- [31] Richard J. Tomsett, Daniel Harborne, Supriyo Chakraborty, Prudhvi K. Gurram, and Alun David Preece. 2019. Sanity Checks for Saliency Metrics. ArXiv abs/1912.01451
- [32] Yao Rong, Tobias Leemann, Vadim Borisov, Gjergji Kasneci, and Enkelejd Kasneci. 2022. A Consistent and Efficient Evaluation Strategy for Attribution Methods. In Proceedings of the Proceedings of the 39th International Conference on Machine Learning, 2022, Proceedings of Machine Learning Research. PMLR, 18770–18795.
- [33] Dong Nguyen 2018. Comparing Automatic and Human Evaluation of Local Explanations for Text Classification. City.

- [34] Qian Yang, Nikola Banovic, and John Zimmerman. 2018. Mapping Machine Learning Advances from HCI Research to Reveal Starting Places for Design Innovation. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, Montreal QC, Canada. Association for Computing Machinery, Paper 130. <https://doi.org/10.1145/3173574.3173704>
- [35] Yao Rong, Tobias Leemann, Thai-Trang Nguyen, Lisa Fiedler, Peizhu Qian, Vaibhav Unhelkar, Tina Seidel, Gjergji Kasneci, and Enkelejda Kasneci. 2024. Towards Human-Centered Explainable AI: A Survey of User Studies for Model Explanations. *IEEE Trans. Pattern Anal. Mach. Intell.* 46, 4, <https://doi.org/10.1109/tpami.2023.3331846>
- [36] Joshua Gorniak, Yoon Kim, Donglai Wei, and Nam Wook Kim. 2024. VizAbility: Enhancing Chart Accessibility with LLM-based Conversational Interaction. In *Proceedings of the Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, 2024, Pittsburgh, PA, USA. Association for Computing Machinery, Article 89. <https://doi.org/10.1145/3654777.3676414>
- [37] Sina Mohseni, Niloofar Zarei, and Eric D. Ragan. 2021. A Multidisciplinary Survey and Framework for Design and Evaluation of Explainable AI Systems. *ACM Trans. Interact. Intell. Syst.* 11, 3–4, <https://doi.org/10.1145/3387166>
- [38] Sven Coppers, Jan Van den Bergh, Kris Luyten, Karin Coninx, Iulianna van der Lek-Ciudin, Tom Vanallemeersch, and Vincent Vandeghinste. 2018. Intellingo: An Intelligible Translation Environment. In *Proceedings of the Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, Montreal QC, Canada. Association for Computing Machinery, Paper 524. <https://doi.org/10.1145/3173574.3174098>
- [39] Josua Krause, Adam Perer, and Kenney Ng. 2016. Interacting with Predictions: Visual Inspection of Black-box Machine Learning Models. In *Proceedings of the Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, 2016, San Jose, California, USA. Association for Computing Machinery, 5686–5697. <https://doi.org/10.1145/2858036.2858529>
- [40] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer’s Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the Proceedings of the 28th International Conference on Intelligent User Interfaces*, 2023, Sydney, NSW, Australia. Association for Computing Machinery, 491–514. <https://doi.org/10.1145/3581641.3584037>
- [41] Helen Sheridan, Emma Murphy, and Dymrna O’Sullivan. 2025. Human Centered Approaches and Taxonomies for Explainable Artificial Intelligence. In *Proceedings of the HCI International 2024 – Late Breaking Papers: 26th International Conference on Human-Computer Interaction, HCII 2024*, Washington, DC, USA, June 29 – July 4, 2024, Proceedings, Part IX, 2025, Washington DC, USA. Springer-Verlag, 144–163. https://doi.org/10.1007/978-3-031-76827-9_9
- [42] P. C. Sruthi, Zinan Guo, Deming Chu, Zhengyan Chen, and Yongle Zhang. 2024. Demystifying the Fight Against Complexity: A Comprehensive Study of Live Debugging Activities in Production Cloud Systems. In *Proceedings of the Proceedings of the 2024 ACM Symposium on Cloud Computing*, 2024, Redmond, WA, USA. Association for Computing Machinery, 341–360. <https://doi.org/10.1145/3698038.3698568>
- [43] Kati Kuusinen. 2016. Are Software Developers Just Users of Development Tools? Assessing Developer Experience of a Graphical User Interface Designer. Springer International Publishing, City.
- [44] Masitah Ghazali, and Alfian Naufal Ravi Hidayat. 2023. Enhancing the Developer Experience (DX) in Docker Supported Projects. *International Journal of Innovative Computing*
- [45] Abdul Razzaq, Jim Buckley, Qin Lai, Tingting Yu, and Goetz Botterweck. 2024. A Systematic Literature Review on the Influence of Enhanced Developer Experience on Developers’ Productivity: Factors, Practices, and Recommendations. *ACM Comput. Surv.* 57, 1, <https://doi.org/10.1145/3687299>
- [46] Daniel Pinho, Ademar Aguiar, and Vasco Amaral. 2024. Cognitive Patterns for Developer Experience. In *Proceedings of the Proceedings of the 29th European Conference on Pattern Languages of Programs, People, and Practices*, 2024. Association for Computing Machinery, Article 6. <https://doi.org/10.1145/3698322.3698345>
- [47] Awdren Fontão, Arilo Dias-Neto, and Davi Viana. 2017. Investigating factors that influence developers’ experience in mobile software ecosystems. In *Proceedings of the Proceedings of the Joint 5th International Workshop on Software Engineering for Systems-of-Systems and 11th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems*, 2017, Buenos Aires, Argentina. IEEE Press, 55–58.
- [48] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: a study of developer work habits. In *Proceedings of the Proceedings of the 28th international conference on Software engineering*, 2006, Shanghai, China. Association for Computing Machinery, 492–501. <https://doi.org/10.1145/1134285.1134355>
- [49] Mauro Cherubini, Gina Venolia, Robert A Deline, and Amy J. Ko. 2007. Let’s go to the whiteboard: how and why software developers use drawings. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*
- [50] Marco Torchiano, Federico Tomassetti, Filippo Ricca, Alessandro Tiso, and Gianna Reggio. 2013. Relevance, benefits, and problems of software modelling and model driven techniques-A survey in the Italian industry. *J. Syst. Softw.* 86, 8, <https://doi.org/10.1016/j.jss.2013.03.084>
- [51] G. Johnson, and Mario Verdicchio. 2023. Ethical AI is Not about AI. *Commun. ACM* 66, 2, <https://doi.org/10.1145/3576932>
- [52] Beatrice Vincenzi, Simone Stumpf, Alex S. Taylor, and Yuri Nakao. 2024. Lay User Involvement in Developing Human-centric Responsible AI Systems: When and How? *ACM J. Responsib. Comput.* 1, 2, <https://doi.org/10.1145/3652592>

APPENDICES

A Focus Group Plan and Question Guide

A.1 Objective

To gather insights into the behaviors, challenges, and needs of developers across different skill levels (junior, intermediate, senior) during the LLM development lifecycle, focusing on key factors like mental models, efficiency, visualization, and understandability.

A.2 Interview Plan

Participant Recruitment

Recruit 30 developers (10 each from junior, intermediate, and senior levels) with experience in LLM development or related workflows.

Format

Group interviews, 45–60 minutes per session, conducted via video conferencing.

Structure

Introduction (5 minutes): Brief overview of the study, purpose of the interview, and confidentiality agreement.

Core Questions (35–45 minutes): Focus on specific stages of the LLM lifecycle and key DX factors.

Wrap-Up (5 minutes): Summarize responses and ask for additional thoughts or suggestions.

Recording and Documentation

With participant consent, record interviews for transcription and analysis. Take notes during each session.

Analysis Method

Perform thematic analysis to identify recurring patterns, insights, and challenges across different skill levels.

A.2.1 Question Guide. We used structured approach below to ensure consistency across interviews while leaving room for participants to share unique insights.

Q1 Can you briefly describe your role and experience in LLM development?

Q2 What stage(s) of the LLMops lifecycle do you primarily work on?

Q3 How do you approach preparing datasets for LLM training?

Q4 What challenges do you face when assessing data quality or detecting issues in datasets?

Q5 What tools or features would make this stage more efficient or intuitive?

Q6 How do you monitor model training progress? Are there specific metrics you rely on?

Q7 What difficulties do you encounter when debugging training-related issues (e.g., overfitting, gradient instability)?

Q8 What type of real-time feedback would help you during this stage?

Q9 What are your main concerns when deploying LLMs into production?

Q10 How do you ensure the quality of the model's outputs?

Q11 Would tools for real-time intervention or output correction be helpful? If so, how?

Q12 How do you track model performance over time?

Q13 What challenges do you face in detecting data drift or optimizing models in production environments?

Q14 How important is clear version history for your workflow? What features would improve this?

Q15 How do you assess the understandability of the tools or systems you work with?

Q16 Are visualizations (e.g., graphs, heatmaps) effective for your tasks? If not, what improvements would you suggest?

Q17 What aspects of current tools impact your efficiency, positively or negatively?

Q18 How do you think tools can better cater to your specific skill level (junior, intermediate, senior)?

Q19 Are there any specific features or design principles that you find particularly helpful or lacking?

Q20 What features or improvements would you prioritize in LLM development tools?

Q21 Do you have any additional thoughts on how to enhance the developer experience across the LLM lifecycle?

Thank you for your time and valuable input.

A.3 Survey and Questionnaire

A.3.1 Objective. To evaluate the correlation between four design principles and key developer experience factors (Mental Models, Efficiency, Visualization, Understandability) across the LLM development lifecycle using a detailed rating scale.

A.3.2 Survey Instructions. Here's the survey questionnaire with a -1 to 1 scale divided into 10 intervals (steps of 0.2). This approach allows for more granularity in assessing the relevance of design principles to DX factors.

For each design principle and DX factor pairing, please rate the degree of correlation using the scale below. Choose the value that best represents the level of relevance.

Scale Descriptions are as below, please feel free to rate and add comments to explain your reasoning.

- -1.0 = Strongly irrelevant
- -0.8 = Moderately irrelevant
- -0.6 = Slightly irrelevant
- -0.4 = Minimally irrelevant
- 0.2 = Barely irrelevant
- 0.0 = Neutral
- 0.2 = Barely relevant
- 0.4 = Minimally relevant
- 0.6 = Slightly relevant
- 0.8 = Moderately relevant
- 1.0 = Strongly relevant

A.3.3 Survey Questions. Q1 What is your role in LLM development or tool design?

Q2 How many years of experience do you have in designing or developing tools for LLM workflows?

- 0-2 years
- 3-5 years
- 6+ years

Q3 Which design principle do you believe has the strongest overall impact on developer experience? Please explain.

Q4 Are there any DX factors not covered in this survey that you think are important?

Q5 Have you observed any conflicts or trade-offs between the design principles? If so, please provide examples.

Q6 How do you think these design principles could be improved to better align with developer needs?